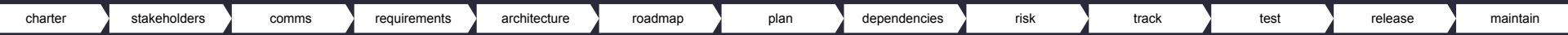
TPM Framework

Stan Toman

Presentation Structure:



code sample @ https://github.com/st-mn/chainlink-sample-datafeed-process-e2e-brief

Intro to this presentation for reviewers

How to view this presentation

This presentation should be viewed as a delivery framework with draft artefacts and processes to be tailored together with stakeholders. I focused more on structure and less on estimation.
It's based on assumptions and ***not all proposed processes might need to be implemented****, depending on the state of current Chainlink processes.*
We shall see what brings best value and what makes sense.

Plan to deploy first Data Feed 2.0 into TestNet

The goal of this take home exercise is to create a program plan deploying first Data Feed 2.0 into TestNet (goal of my Q2 MVP plan). I focus on my daily work with engineers and stakeholders using agile scrum ceremonies.

Program delivery framework

Beyond the Q2 MVP plan, this presentation outlines my full program framework, for cross-functional work with engineering (QA, Data, Operations, GTM), EMs, PDMs, and VPs, enabling our organization to deliver Data Feeds 2.0. Service to testnet.

Drafted program artefacts

This presentation includes drafts of Data Feeds 2.0 program delivery artefact which I will use to drive the initiative. All are extracted from my JIRA/Confluence - hope to migrate it to Chainlink JIRA in future:).

New feed onboarding workflow

Additionally, I propose to create a continuous workflow for new data feeds onboarding, outlined on the ***Maintenance*** *slide, which can be done parallel to the Data Feed 2.0 architecture delivery work.*

I might have gone a bit extra here :), but I am very passionate about Chainlink, and hope to bring structure enabling everyone to succeed and deliver highest quality Data Feeds 2.0 service.

Presentation Structure

- *Program Charter*
- *Stakeholders*
- *Communication*
- *Requirements*
- *Architecture*
- *Roadmap*
- *Detailed Plan*
- *Dependencies*
- *Risk*
- *Tracking*
- *Testing*
- *Release*
- *Maintenance*
- *FAQ*

Program Charter

Full Detail @ https://stmn.atlassian.net/wiki/spaces/~6068ab0e9bc49b00701d1b0f/pages/3145771/Data+Feeds+2.0+Program+Charter

Goal of the program is to deploy new Data Feeds 2.0 solution running in performant and pull based architecture of Data Streams.

Purpose of the Program

- Program leads all **efforts** of implementing and **launching** of Data **Feeds** 2.0 on new **architecture**.

Problem Statement

- Current Data Feeds 1.0 architecture **doesn't** fully **scale** to handle **future growth**.

Objectives

- Improving **scalability** and performance.
- Expanding **depth** of data.
- **Enabling** less **liquid** users.
- **Unlocking** more **RWA**.

Scope

- MVP **launch** first Data Feed 2.0 on **testnet**.
- Fully **launch** Data Feed 2.0 **service on testnet**.

Out of Scope

- Maintenance or development of **Data Feeds 1.0**

Success Criteria

- New Data Feed 2.0 solution must be **operational on chain**. Transactions measured **on etherscan. Other performance KPIs**. (later slides)

Roadmap and Timeline

- **Assuming** team capacity **40** SP per **2-week** sprint.
- Q1-Q3/Q4 2024 (to be aligned).

Stakeholders

- Program is cross-functional with **multiple** teams- Data, Operations, QA, GTM and stakeholders.

Communication Plan

- Whole working group is engaged and cooperates by defined **communication plan**.

Risks and Dependencies

- Risks and Dependencies are closely **managed** by designed **framework**.

Delivery Approach

- We follow **agile** delivery approach with **Scrum ceremonies** and **roles**.

Test and Release Management

- In order to ensure **highest quality** of our services we follows **rigorous test** and **release procedures**.

(See below slides for details of each of these sections)

Stakeholder Management

Full Detail @ https://stmn.atlassian.net/wiki/spaces/~6068ab0e9bc49b00701d1b0f/pages/3211266/Stakeholder+Management

- Data Feeds 2.0 Program is cross-functional and involves **multiple stakeholders** which are managed by following **principles**:
  - Stakeholders provide **sign off** on program **artifacts** and **participate** in specific **initiatives according** to their roles in **RACI**
  - This **ensures** constructive **dialog**, collaborative **decision**-making and **ownership**

- Program has **4 core** teams - **Data**, **Operations**, **QA, GTM**
- Program **also cooperates** with **Data Streams PoC** especially during design
- Program **also cooperates** with **Security** team
- Other roles in program: Product Management VP (**PDVP**), Engineering Manager (**EM**), Product Manager (**PDM**) and Program Manager (**TPM**)
- RACI should be expanded and tailored in alignment with the Chainlink engineering organization

	PDVP	EM	PDM	TPM	Data Team	QA Team	Operations Team	Security Team	Data Streams Tech Lead	GTM Sales Team
Product Roadmap	I	r	A	R	C	C	C	C	C	C
Program Timeline	I	r	C	A R	C	C	C	I	C	C
Requirements	I	C	C	A R	C	C	C	C	C	C
Infrastructure Setup	I	C	C	A	r	C	R	C	C	-
Program Risks	I	C	C	A R	C	C	C	C	C	-
Architecture Design	I	r	C	A	R	C	r	C	C	-
Implementation	I	r	C	A	R	C	r	I	C	-
Testing	I	C	C	A	r	R	r	C	C	-
Release	I	r	C	A	r	r	R	I	C	C
Continous New Feed Onboarding	I	r	C	A	R	r	r	I	C	r
...										

Communication Management

Full Detail @ https://stmn.atlassian.net/wiki/spaces/~6068ab0e9bc49b00701d1b0f/pages/3211266/Stakeholder+Management

Cadence	Channel	Audience	Purpose
Daily	Scrum Standup	Scrum Teams and Dependencies	Daily work sssignments and removal of impediments helping to enable Eengineering teams
Weekly	TPM:PDM:EM 1-1-1		Align on wbs and timeline and backlog grooming
Bi-Weekly	Sprint Planning Sprint Review and Retro	Scrum Teams and Dependencies	Review of completed sprint and Planning of next sprint
Bi-Monthly or Async	Program Governance Meeting	All Steakholders	Alignment of all stakeholders on direction of program and agreement on necessary changes
Quarterly	Newsletter	All Steakholders	Update of all stakeholders on delivered and planned benefits and releases
Yearly	24 Month Product Vision 24 Month Organization Forecast	All Steakholders	Products and services future vision and forecast of resources
Async	Feedback Loop Improvement Survey	All Steakholders	Continuous feedback loop with all stakeholders for improvement of programs, structures and processes
Async	Technical Engineering Reviews	Scrum Teams as required	Resolving and consensus on potential technical disagreements

- **Communication** plan **defines** cross-team communication **channels** to avoid **miscommunications** and **ensure transparent delivery**. Specific use-cases are:
  - Periodically **reconfirm** cross-team **dependencies**
  - Newsletter with **benefits** and releases
  - All scrum **ceremonies**
  - **Weekly** sync with EM and PDM
  - **Ad hoc** technical reviews
- Single **source** of **truth** for all **repositories**

Repository Link	Content	Owner	Contributors
Github Repositories	Source Code	Data Team	TPM
Charter	Complete Project Charter in Confluence	TPM	All Stakeholders
Requirements	All Project Requirements in Confluence JIRA	TPM	All Stakeholders
Roadmap	Project Roadmap in Confluence	TPM	All Stakeholders
Timeline	Project Timeline in JIRA	TPM	All Stakeholders
Stakeholders	Project Stakeholders in Confluence	TPM	All Stakeholders
Communication	Project Communication Plan in Confluence	TPM	All Stakeholders
Risk Register	Project Risks Managed in JIRA	TPM	All Stakeholders
Test Management	Test Cases in Confluence	TPM	Data+QA Teams
Release Management	Canary and Full Release Planning in JIRA	TPM	Platform Team
User Documentation	End User Documentation on Public Web	TPM	All Stakeholders

Requirements Management and Traceability

Full Detail @ https://stmn.atlassian.net/wiki/spaces/~6068ab0e9bc49b00701d1b0f/pages/3178507/Data+Feeds+2.0+TestNet+Requirements

Each requirement is traceable end to end as follows:

Business Requirement [BR-XX] <-> (Non)Functional Requirement [FR-XX] <-> JIRA WBS Epic [EP-XX] <-> JIRA WBS Work Package Task [WP-XX] <-> JIRA TEST Report [TE-XX] <-> JIRA Release Version [RV-XX]

Business Requirement 1: Improving Scalability [BR-1]

- New architecture based on pull based high-performance **datastreams design blueprint**.
- Aggregation of feeds into **one cache** as opposed to multiple contracts for each feed in past.
- The new feeds will run on upgraded **OCR 2.0** protocol.

Business Requirement 2: Increasing Depth Of Data [BR-2]

- Extend the data feeds with **grouping** where each feed can deliver collection of data **types eg. precious metals data**, commodity data.
- Include **on chain billing** not only with a **fixed** prize but also with **bid/ask** option.
- Add updated **price on demand** and **metadata** which can all be requested in a **single contract call**.

Business Requirement 3: Enabling Less Liquid Users [BR-3]

- Offer new **tiered oracle** solution while keeping cryptographic guarantees and security standards.
- This will enable long tail of less liquid assets to benefit from Chainlink data feeds and usual **volume/liquidity boost**.

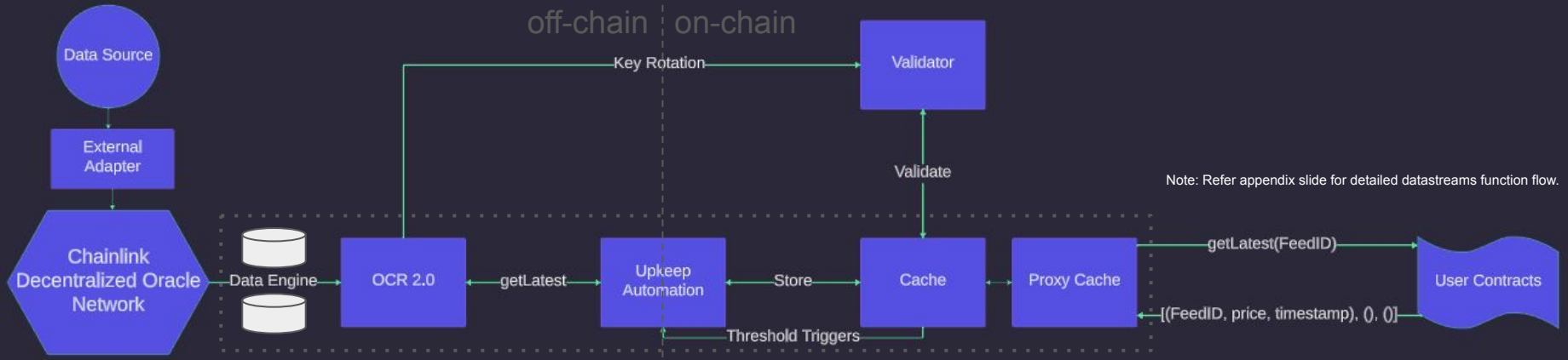
Business Requirement 4: Unlocking More Real World Assets [BR-4]

- Currently cover money markets, commodities and ETFs.
- Aim to add more protocols for **luxury retail**, **real estate** and other industries in demand.

BR	FR	P	Reqs Status	Jira Epic	Jira WP	Jira Test	Jira Rel
BR-1 Improve Performance and Scalability	FR-1.1 DataStream Pull Based Architecture FR-1.2 Fully Supports OCR 2.0 Protocol FR-1.3 Caching Allows Chain Agnostic Scalability	HIGH	[DRAFT]	[EP-XX]	[WP-XX]	[TE-XX]	[RV-XX]
BR-2 Increasing Depth Of Data	FR-2.1 Grouping of Feeds FR-2.2 On Demand Price Updates FR-2.3 On Chain Billing	HIGH	[DRAFT]	[EP-XX]	[WP-XX]	[TE-XX]	[RV-XX]
BR-3 Enabling Less Liquid Users	FR-3.1 Tiered Oracle Solution for Less Liquid Users	HIGH	[DRAFT]	[EP-XX]	[WP-XX]	[TE-XX]	[RV-XX]
BR-4 Unlocking More Real World Assets	FR-4.1 Onboard Luxury Retail Assets FR-4.1 Onboard Real Estate Assets	HIGH	[DRAFT]	[EP-XX]	[WP-XX]	[TE-XX]	[RV-XX]

Architecture Design

Full Detail @ https://stmn.atlassian.net/wiki/spaces/~6068ab0e9bc49b00701d1b0f/pages/3145784/Data+Feeds+2.0+Design



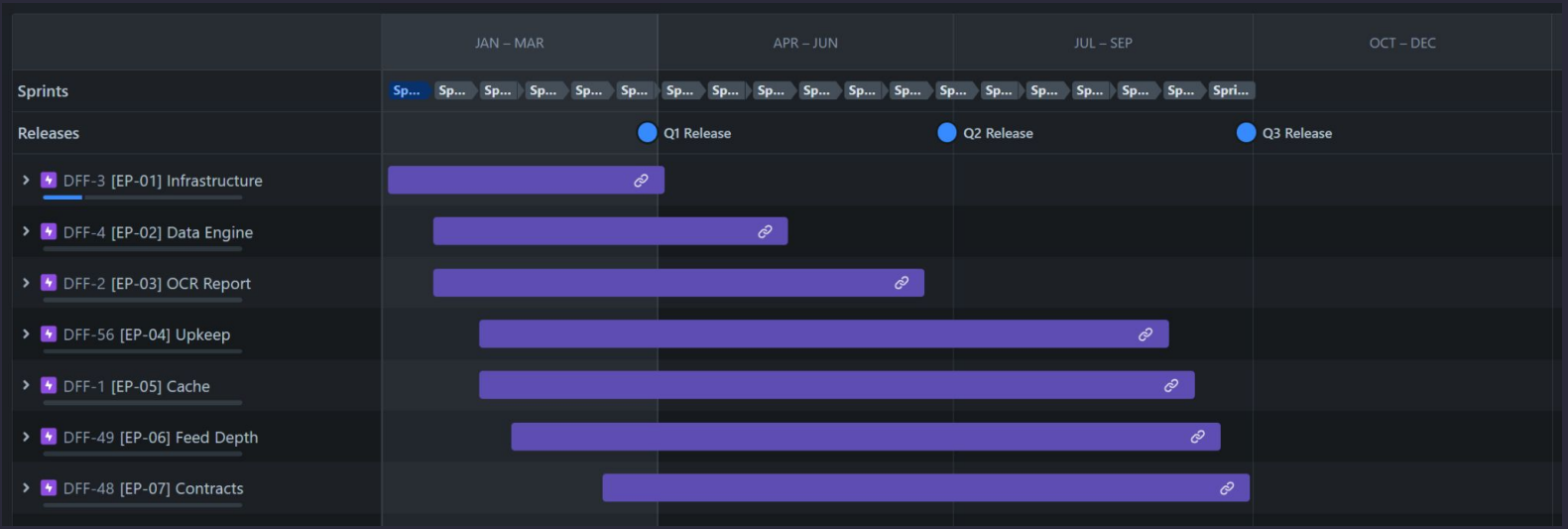
Approach defining WBS together with teams:

1. Design is **based** on current **DataStreams** (DS) **architecture**
2. Clearly **define** Data Feeds 2.0 (DF2) **Reqs** and **Kpi**
3. **Overlay** DF2 **requirements** over DS **architecture**
4. **Gap analysis** identifies difference between DS and DF2
5. **List** DF2 required **high level features** to be implemented
6. **Break** down high level features to **sprint long tasks**
7. Identify **dependencies** between tasks
8. **Align dependencies** relationships and **conflicting priorities**
9. Identify **risks** and **mitigations**
10. **Prioritize** risks
11. Define **plan** and tracking
12. **Publish WBS**
13. **Assign** dependencies and tasks to teams
14. **Kickoff** and continuously improve

Implementation Roadmap - Milestones and Success Criteria

Full Detail @ https://stmn.atlassian.net/jira/software/projects/DFF/boards/6/timeline

- **Q1** focus is on **design** and setting up **infrastructure**
- **Q2** focus is on getting first data **feed on testnet**
- **Q3** focus is on achieving data feed **depth and performance**
- Table with each **milestone** has success criteria **validated** by **test**



Quarterly milestones [M] and success criteria [K] with validating tests (TE):

Q1 30-Mar-2024	**Q2 MVP** 30-Jun-2024	Q3 30-Sep-2024
[M1.1] Infrastructure Setup Done [M1.2] Basic OCR Data Engine Done	[M2.1] Upkeep Caching and Proxy Done [M2.2] First Basic Data Feed 2.0 on Testnet Done	[M3.1] Full Depth Data Feed 2.0 Done [M3.2] Data Feed 2.0 Achieves Low latency
[K1.1] Data DON is running as seen in logs (TE-1.1) [K1.2] OCR reports are generated (TE-1.2)	[K2.1] Cache is updated with Upkeep (TE-2.1) [K2.2] First Basic Data Feed 2.0 is on etherscan (TE-2.2)	[K3.1] Full Depth of Data Feed 2.0 tested (TE-3.1) [K3.2] Data Feed 2.0 latency below spec. threshold (TE-3.2)

For contingency we sprint towards Internal deadline Q3 2024 and external public deadline Q4 2024.

Detailed Implementation Plan WBS and KPI Checks

Full Detail @ https://stmn.atlassian.net/jira/software/projects/DFF/boards/6/timeline

ver	Infrastructure [EP-01] Feature Epic Deliverables	Data Engine [EP-02] Feature Epic Deliverables	OCR Report [EP-03] Feature Epic Deliverables	Upkeep [EP-04] Feature Epic Deliverables	Cache [EP-05] Feature Epic Deliverables	Feed Depth [EP-06] Feature Epic Deliverables	Contracts [EP-07] Feature Epic Deliverables	Key Performance Indicator Checks KPI Measures
v0.1	-[WP-11] Dev Infra Setup with DataFeed	-[WP-20] Engine Requirements and Design						-[KP-01] Design signoff -[KP-02] Check Data DON Up
v0.2	-[WP-12] Infra integrated with DataEngine Interface	-[WP-21] Instance Feed Report Plugin OCR2.0	-[WP-31] Sample Report Generated					-[KP-03] Check Data Engine Up -[KP-04] Check Sample OCR report arrived
v0.3	-[WP-13] Infra integrated with Automation	-[WP-22] Implement libocr generation routines	-[WP-32]Cryptographically signed Report	-[WP-41] Basic Automation Keeper setup	-[WP-51] Cache Infra Setup			-[KP-05] Check Cache Infra Up -[KP-06] Check Upkeep triggers running
v0.4	-[WP-14] Infra integrated with Caching + Validator	-[WP-23] Implement libocr transmit routines	-[WP-33] Validated Report	-[WP-42] Detailed Triggers	-[WP-52] Proxy Infra Setup	-[WP-61] Basic Feed Flow	-[WP-70] Report Validator Contract	-[KP-07] Check Proxy Infra Up -[KP-08] Check Basic feed arrived on proxy
v0.5 Q2 MVP	-[WP-15] Full infra up and running e2e	-[WP-24] Performance and **Security** tested Data Engine	-[WP-34] Fully Valid OCR 2.0 Report	-[WP-43] Automation Register Testnet	-[WP-53] Single cache+proxy persistence tested	-[WP-62] Feed on Ropsten Testnet	-[WP-71] Verified OCR report contracts	-[KP-09] Check E2E flow source-contract verified feed on Testnet MVP
v0.6		-[WP-26] Optimized Data Engine	-[WP-35] Performance tested OCR 2.0 Report	-[WP-44] Expanded Keepers logic	-[WP-54] Stress tested cache+proxy	-[WP-63] On Demand Update Feed	-[WP-72] On demand updated contracts	-[KP-10] Check Monitoring events firing -[KP-11] Check On Demand updated contracts
v0.7				-[WP-45] Multichain Keepers	-[WP-55] Optimized cache+proxy	-[WP-64] Billing Feed	-[WP-73] Billing Contracts Basic and Premium	-[KP-12] Check Keepers availability 99.99% -[KP-13] Check Onchain billing functional
v0.8				-[WP-46] Optimized Multi Chain Keepers	-[WP-56] Multi Chain cache+proxy	-[WP-65] Priority Gas Feed	-[WP-74] Priority Feed Contracts	-[KP-14] Check Multi Chain cache+proxy Up -[KP-15] Check Priority feed contracts
v0.9					-[WP-57] Optimized Multi Chain cache+proxy	-[WP-66] Stress Tested Feed Groups	-[WP-75] Performance tested Contracts	-[KP-16] Check Robust Feed Groups -[KP-17] Check Contract Low Latency

Dependencies Management

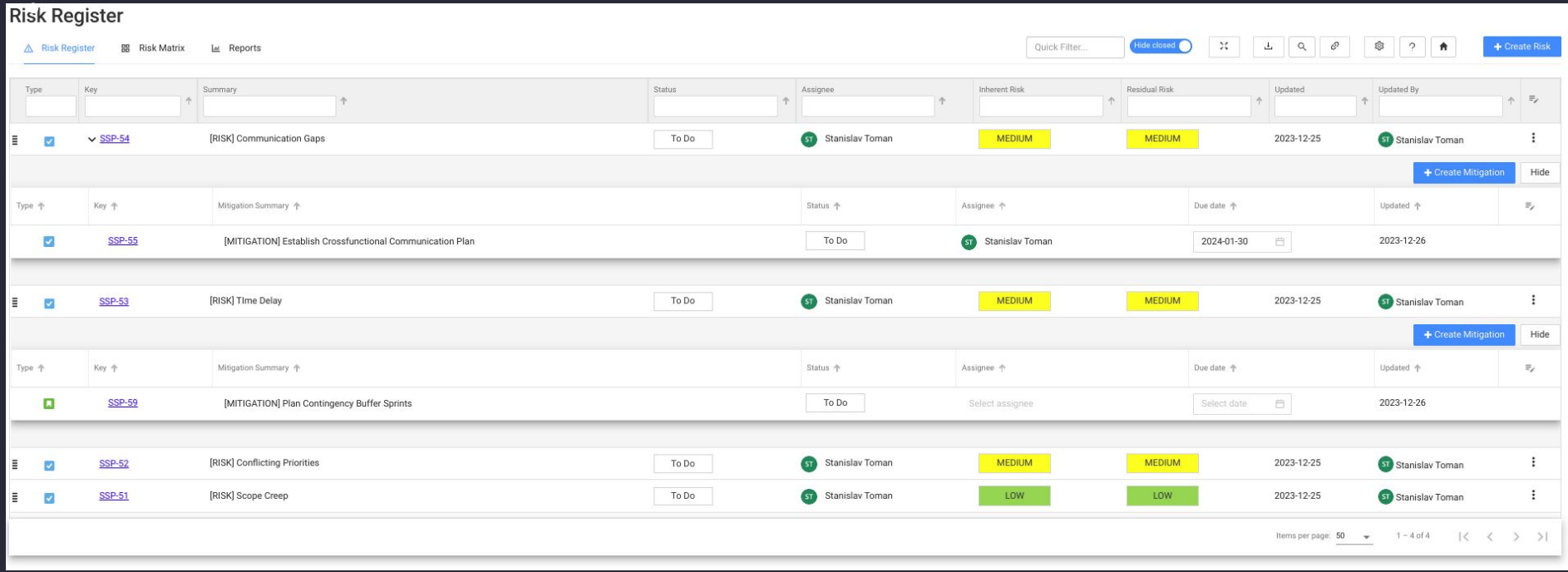
Full Detail @ https://stmn.atlassian.net/jira/plans/1/scenarios/1/dependencies
https://stmn.atlassian.net/jira/software/projects/SSP/apps/8ff417b0-6ab5-4218-bc93-39a196bc62ab/d54506d0-023b-4c33-bc5e-7a39ac462f40

- I work with teams to **identify** and **manage** all **dependencies** in Data Feeds 2.0 Program involving all teams and stakeholders
- All dependencies are **agreed** with respective **owners** and clearly **assigned**
- Dependencies are **periodically reconfirmed** to ensure **no delays** or minimal deviations
- **Competing** dependency priorities are decided according to agreed **prioritization** (eg. based on **business** criticality, **time** criticality, **ROI**)
- Dependencies are **mapped** and tracked in **JIRA** and any changes are **automatically reflected** into program **timeline**
- **Delays** are identified as **early** as possible in every **sprint review**
- **Contingency** plan is used to **compensate** for **delays** and to get whole program back **on track**

Risk Management

Full Detail @ https://stmn.atlassian.net/plugins/servlet/ac/com.boja.riskregister.addon/boja-risk-overview?project.key=FEED&project.id=10006

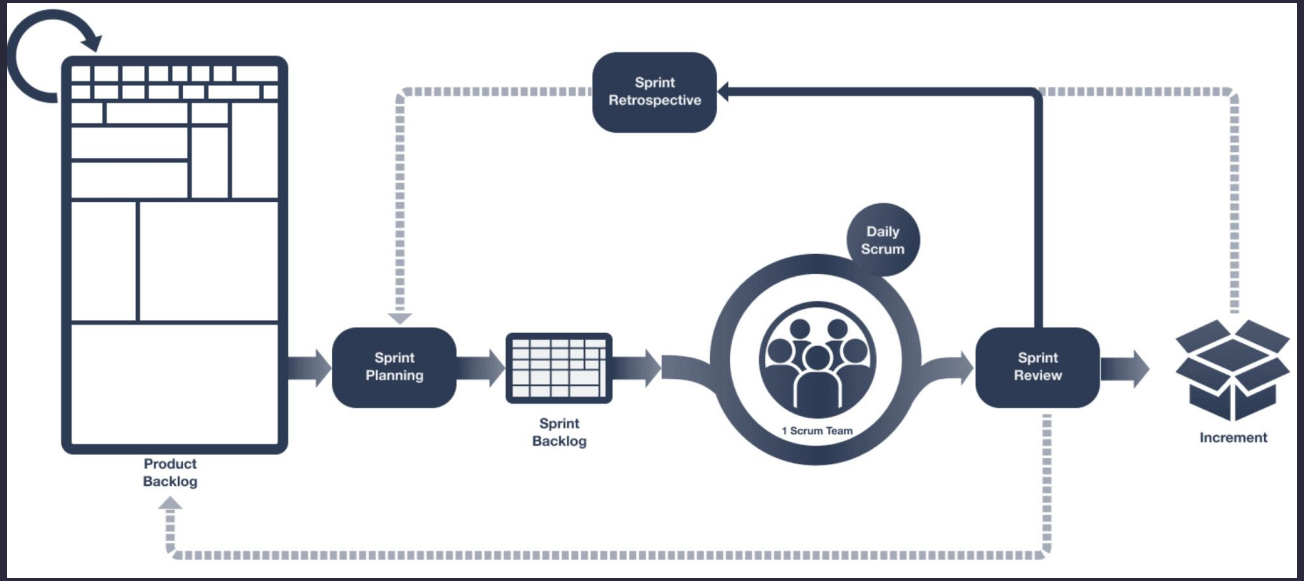
- I work with teams to **identify** all **risks** in the program and **implement mitigations** of these cross-functionally **involving** all **stakeholders**
- **Facilitated tracking** of risks and mitigations in **JIRA Risk Register**
- All **risks** are **periodically** reviewed, assessed, **updated** and communicated **transparently**
- **Risk** Management is **continuous** process followed in **whole** duration of the **program**

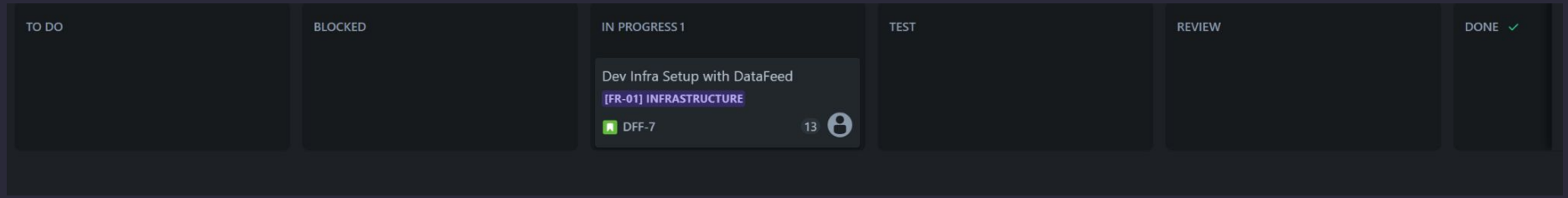


Implementation Work and Tracking

Full Detail @ https://stmn.atlassian.net/jira/software/projects/SSP/boards/6

- I drive all **scrum ceremonies** - sprints planning, review, retro and backlog grooming.
- Actively participate in **daily scrums**. Focusing on **impediments** by facilitated problem-solving, identifying and **removing obstacles** hindering progress.
- Provide **proactive support** by offering guidance, **resources**, and connections to **specialists.**
- **Publicly** acknowledge **achievements** and **motivate** teams.
- **This fosters** a culture of **open communication** through **regular** team **meetings**, one-on-ones, and feedback.

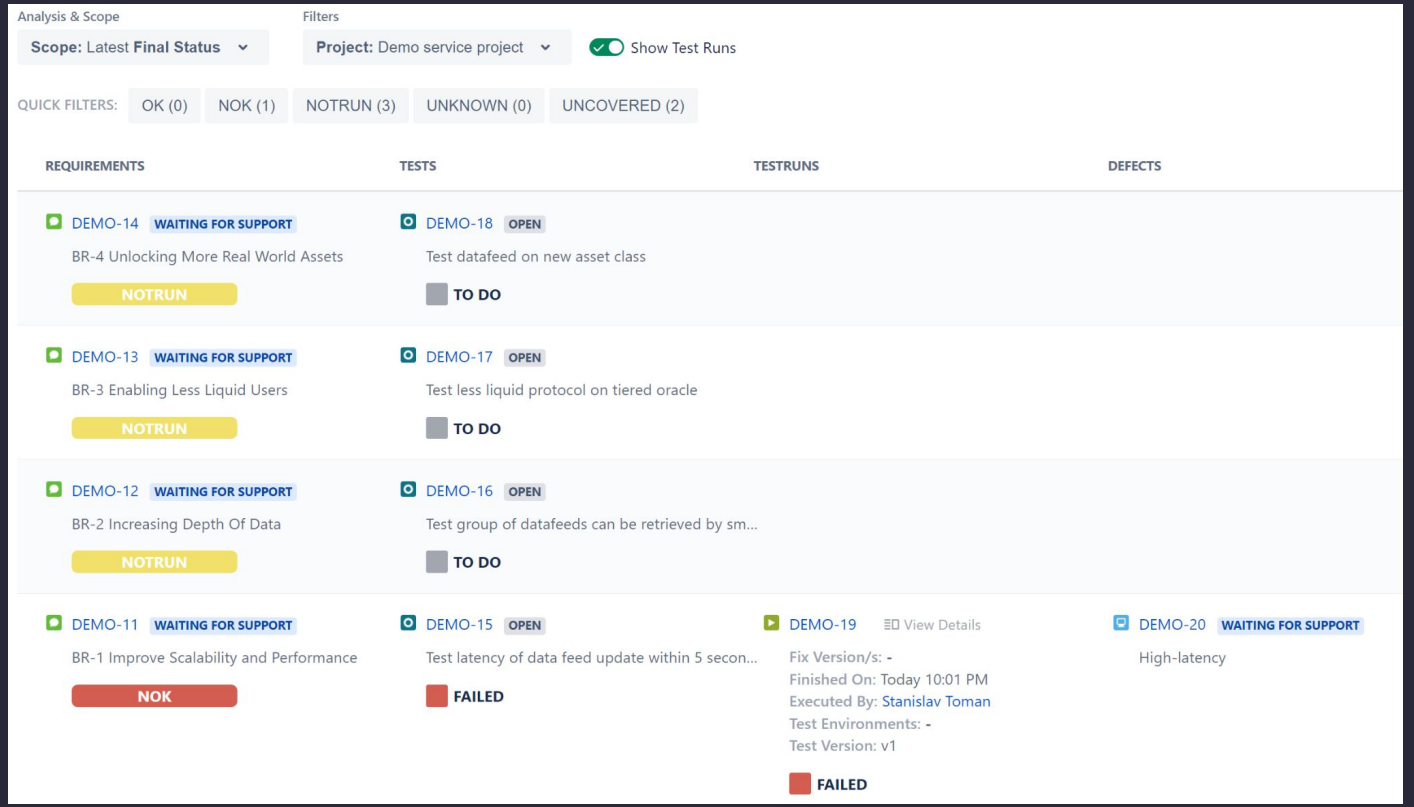




Test Management

Full Detail @ https://stmn.atlassian.net/plugins/servlet/ac/com.xpandit.plugins.xray/traceability-report-page?project.key=DEMO&project.id=10005&servicedesk.serviceDeskId=2

- I work with teams to **implement** a process for Centralized **Test Management.**
- QA team **uses** this process in **JIRA** to track **test** cases, defects, **progress**, and **reporting**.
- **Traceability** Matrix leads from each **requirement** to specific **test** case.
- **T**esting is risk-based to **prioritize critical** requirements.
- Each identified **defect** and failed tests are **tracked** and regressively **re-tested**
- Each **release** is **contingent** to **successful** completion of all **test** cases and **signoff**

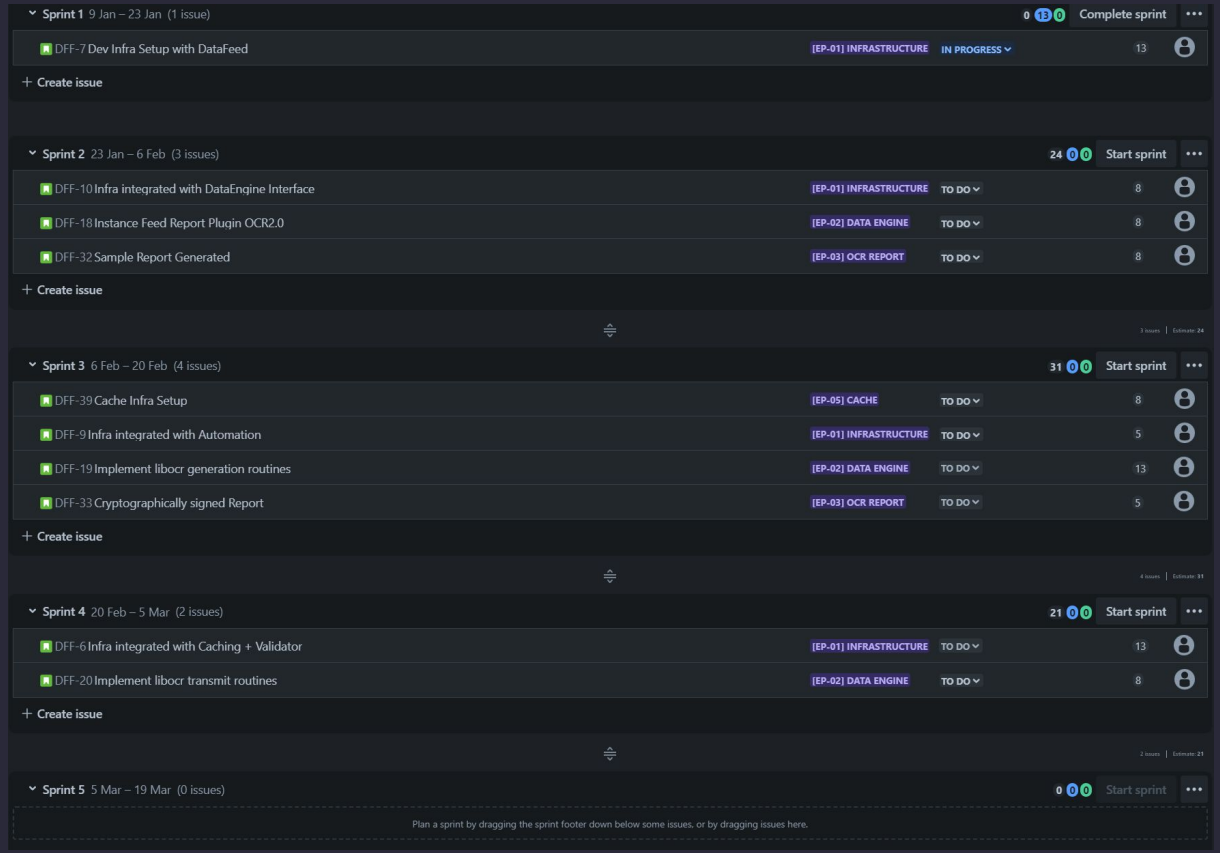


Release Management

Full Detail @ https://stmn.atlassian.net/jira/software/projects/DFF/boards/6/backlog

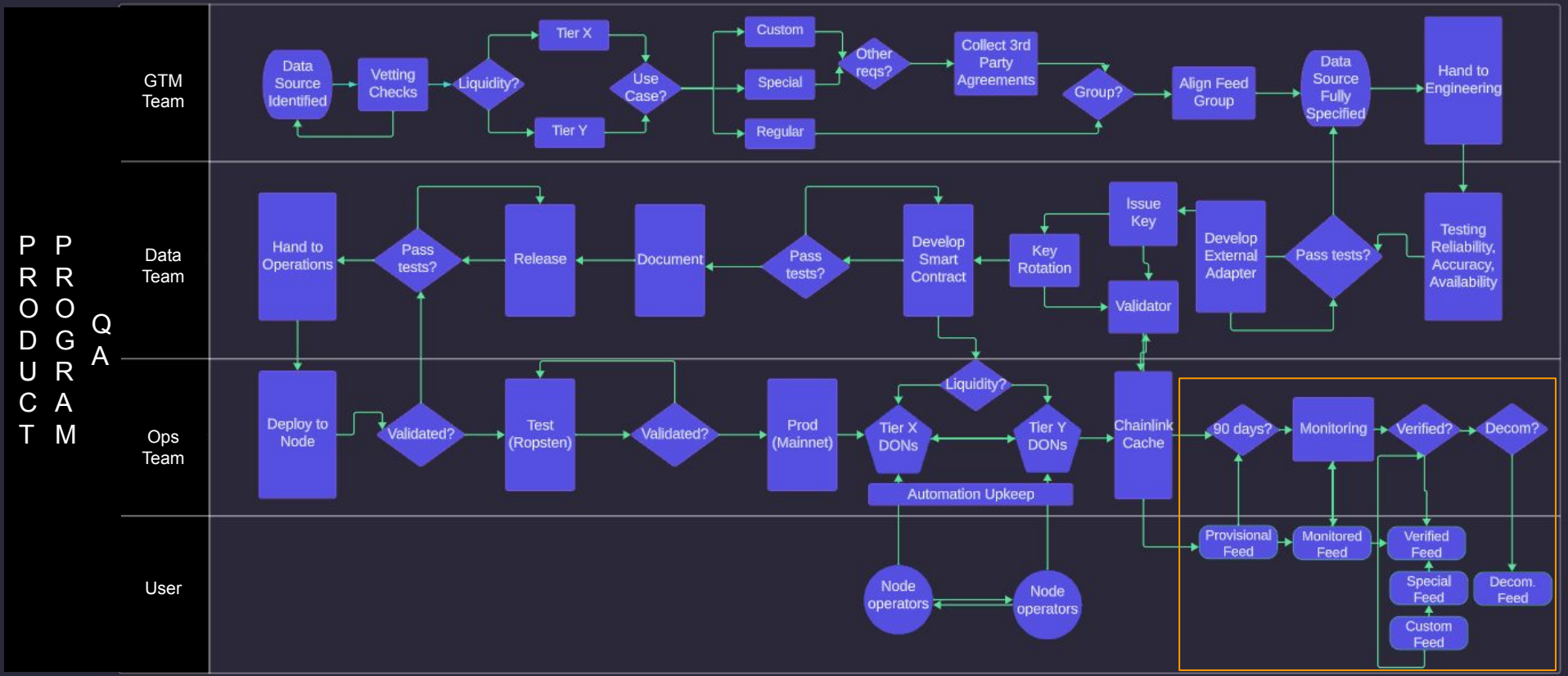
- I work with teams to **plan quarterly releases** of features completed in sprints.
- **Release** is **preceded** by **rigorous** (non-)functional and **testing**.
- We use CI/CD **automation to** streamline deployment processes, improve consistency, and **reduce human error**. JIRA integrated to GH.
- **If needed we have a rollback** plan for fast revert in case of **unexpected issues**.
- For **contingency** every **quarter** has **buffer sprint** to compensate for delays.

Version	Status	Progress
Q3 Release	UNRELEASED	
Q2 Release	UNRELEASED	
Q1 Release	UNRELEASED	



Maintenance and Continuous Feeds Onboarding and Evaluation Process (parallel work)

Full Detail @ https://stmn.atlassian.net/jira/core/projects/FEED/settings/issuetypes/10024/workflow (see slide in appendix for exact JIRA view)



FAQ 1/2

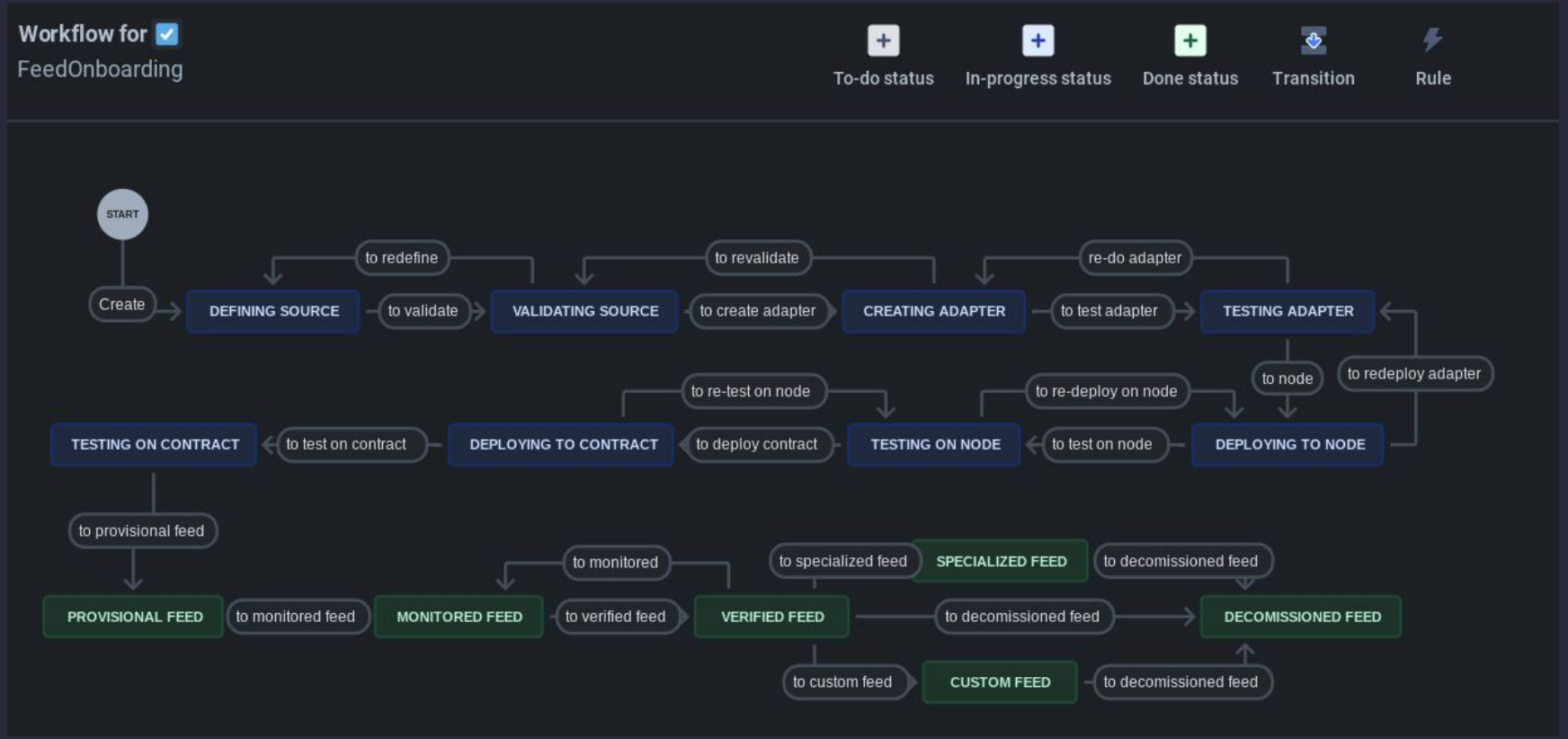
- How do you ensure that no **requirements** are **missed** in requirements analysis?
  - Involve all **stakeholders**. verify and **validate**, manage changes and **continuously revise**.
- How will you **influence** your **stakeholders**?
  - Back up **arguments** with **data**, focus on **shared goals**, clear concise and **effective communication**.
- How will you **provide** any **coaching** along the way?
  - **Individualized** Support, skill-building **workshops**, resource **sharing**, **collaborative** problem-**solving**.
- How will you **overcome** any **resistance** you face?
  - Identify the **root cause**, open communication and **dialogue**, flexibility and **compromise**, highlighting **benefits**, demonstrating **progress**.
- How do you know if the **project** is on **track**?
  - Progress against key **milestones**, performance **metrics** eg. **burndown** and **velocity** charts, **sprint** reviews, periodical **backlog** grooming and **review**.
- How do you **estimate scrum** project timeline **without** knowing **velocity** of the team?
  - Examining similar **historical** projects timeline, consulting **expert** opinion, estimation **techniques** with engineers (poker, buckets, t-shirt sizes), **allowing** to run **initial sprint** for velocity **assessment** and **adjust** timeline **accordingly**.
- How do you ensure **nothing** has been **missed** or overlooked?
  - Thorough and **comprehensive planning** aligned with all stakeholders and teams, **rigorous** and **Iterative** requirements **gathering**.
- How would you manage **competing priorities**?
  - Foster **transparency** through **regular meetings** and status **updates**.
    Establish a **prioritization framework** based on business impact, technical feasibility, and dependencies.
    Encourage cross-team **collaboration** and **drive consensus** by **utilizing** the **prioritization framework**.

FAQ 2/2

- What **other teams** should be **involved** apart from core development teams?
  - **Security** Team. This function might be embedded in multiple teams also.
- How can **stakeholders** provide **sign off**. What would be the **RACI** for this initiative?
  - See RACI in **above slides**.
- How you protect against **Scope Creep**?
  - Proactively **manage expectations** and **enforce** clear **requirements**. **Freeze signoff requirements** in traceability matrix and **govern changes**.
- How do you address **Communication Gaps**?
  - Maintain open **communication channels** and **proactively** seek **feedback**.
- How do you address **Technical Roadblocks**?
  - **Collaborate** with teams to find **solutions** and **mitigate risks**.
- How do you address **Time delay** risk?
  - Every quarter has **buffer sprint** for **contingency**. Utilize **internal** and **external deadlines**.
- How do you **Influence Stakeholders** & Overcome **Resistance**?
  - **Transparency**, Share **data**, risks, and potential **benefits** openly.
  - **Demonstrate value** and showcase the **long-term vision** and impact of the new architecture.
  - **Address concerns** and **tailor** communication to **individual needs**.
- How do you address **Unforeseen Challenges**:
  - Maintain a **culture** of **adaptability** and promote **risk identification** and mitigation, **continuously** assess **potential risks**. Create **contingency plan**.

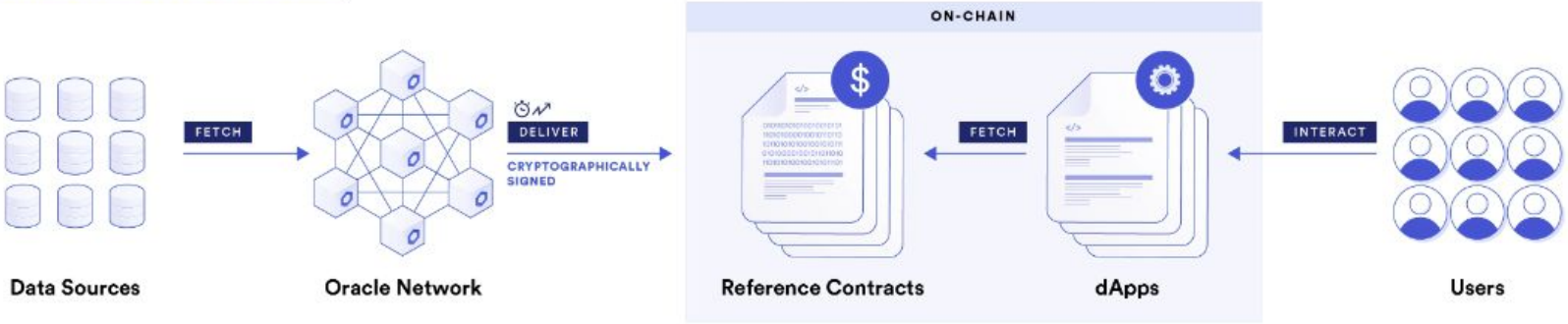
Appendix - Backup slides and references

Data Feeds Onboarding Process Workflow In JIRA stmn.atlassian.net/jira/core/projects/FEED/board

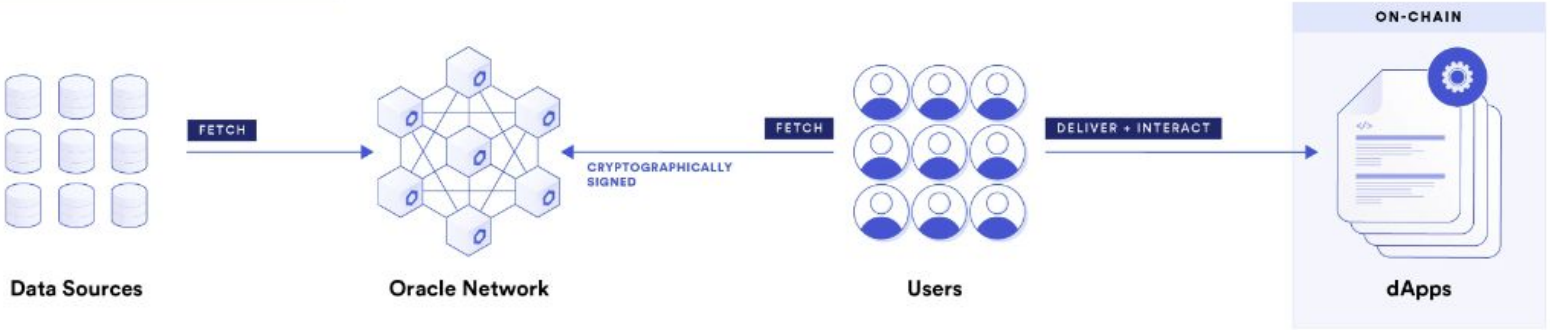


Further Detail : https://stmn.atlassian.net/jira/core/projects/FEED/settings/issuetypes/10024/workflow

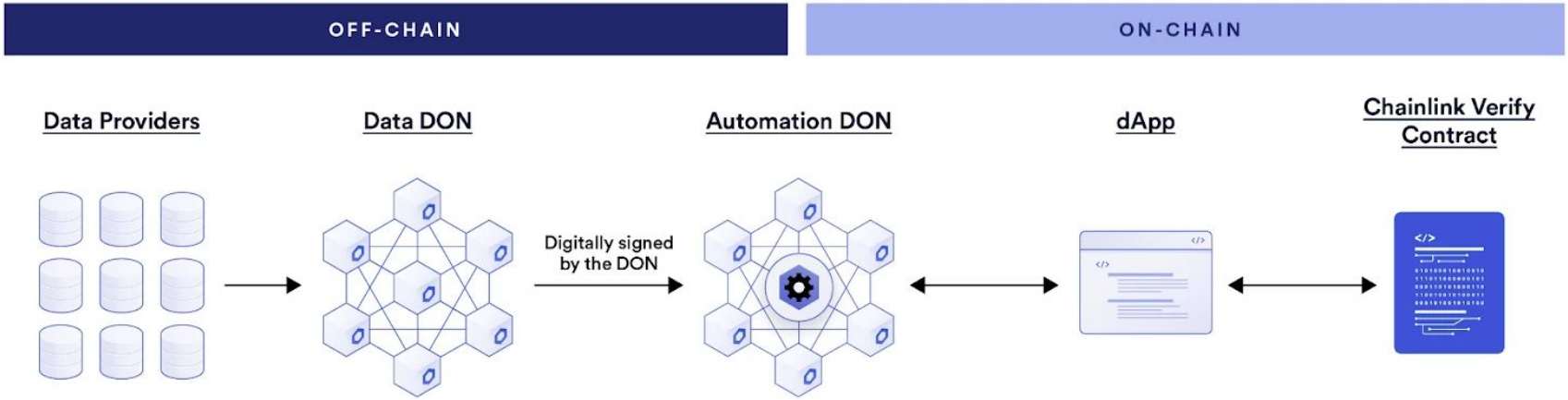
Push-Based Oracles

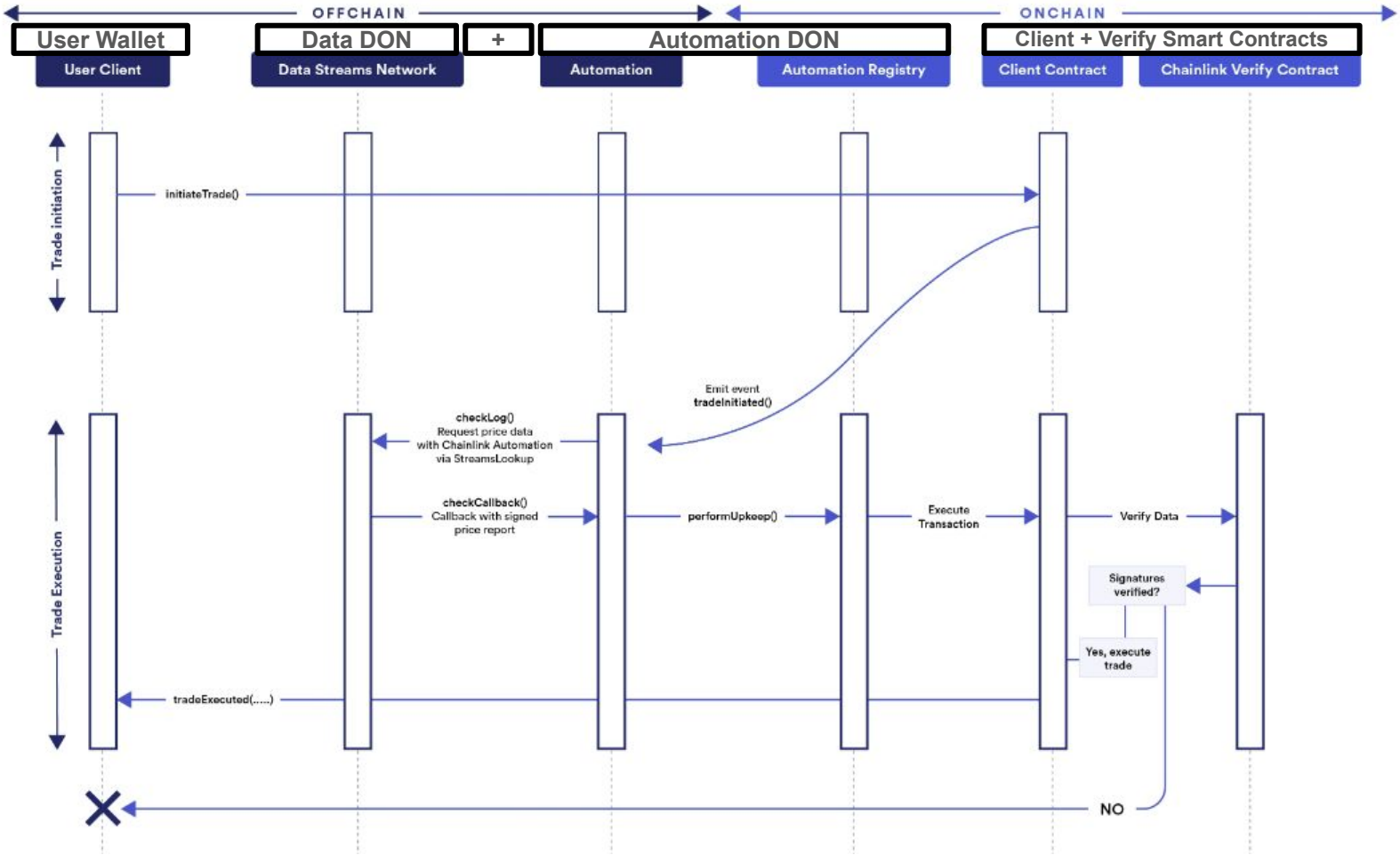


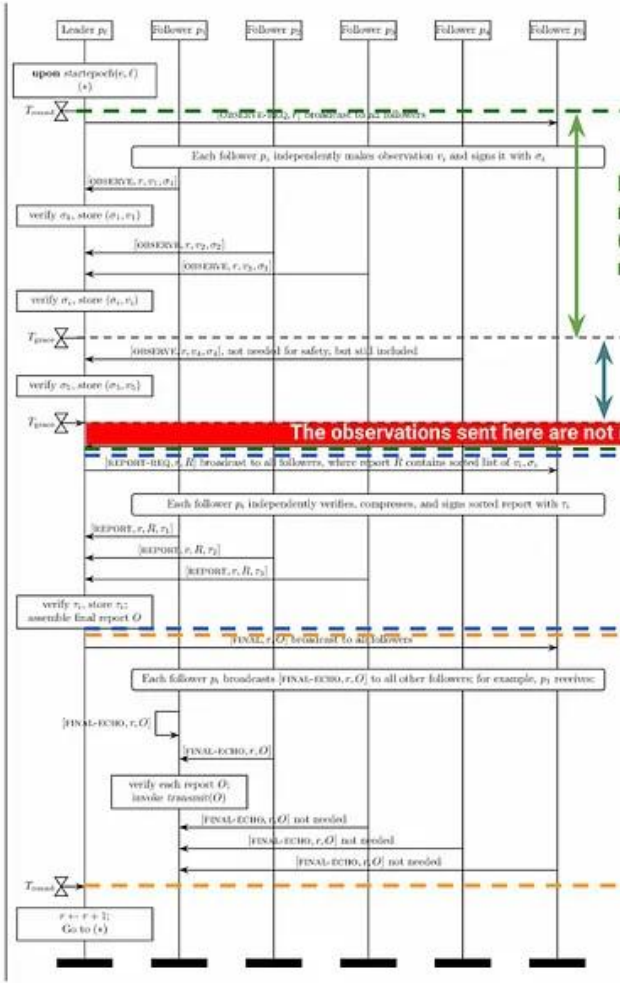
Pull-Based Oracles



Chainlink Data Streams




OFFCHAIN
ONCHAIN
User Wallet
User Client
Data DON
Data Streams Network
+
Automation DON
Automation
Automation Registry
Client + Verify Smart Contracts
Client Contract
Chainlink Verify Contract
Trade initiation
initiateTrade()
Emit event
tradeInitiated()
Trade Execution
checkLog()
Request price data
with Chainlink Automation
via StreamsLookup
checkCallback()
Callback with signed
price report
performUpkeep()
Execute
Transaction
Verify Data
Signatures
verified?
Yes, execute
trade
tradeExecuted(.....)
NO



During this period, the Leader collects the minimum required number of observations (more than 2/3 of the number of p2p network participants)

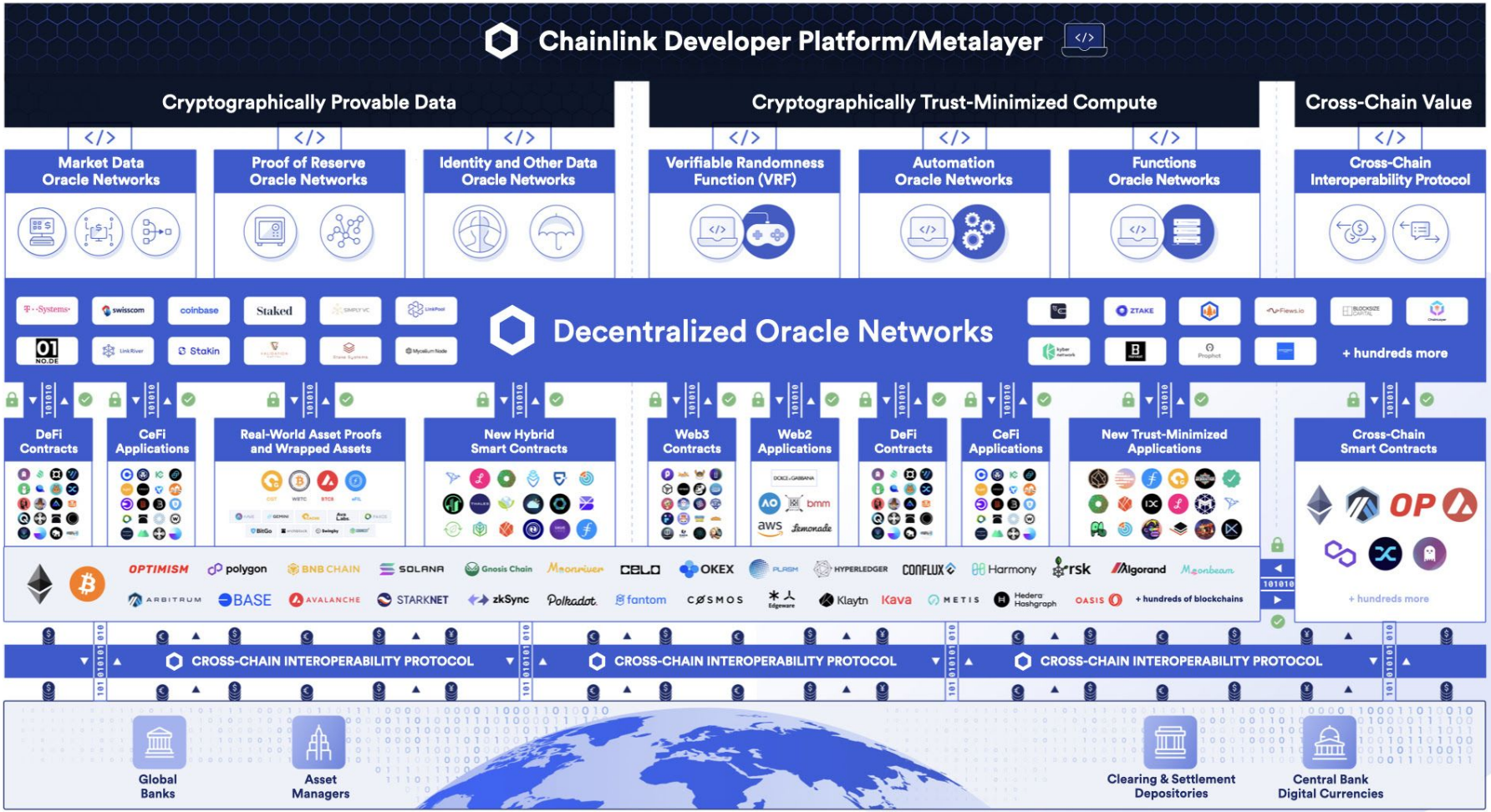
During this period, the Leader gets answers from the other participants who were a little late in sending the observation.

The observations sent here are not included in the oracle report!

Stage 1. The *Leader* requests observations from all participants. Participants send their observations, signed with their private key. The Leader needs to receive observations from more than 2/3 of the network participants. The Leader then generates an "intermediate oracle report".

Stage 2. The Leader sends an "intermediate oracle report" to the network members for verification. They verify it (make sure all the signatures are correct), extract a compressed report containing just the observations and oracle identities, and send back a signature on that.The *Leader* must receive signed reports from more than 1/3 of the network members. The *Leader* then compress all reports and generates a "final oracle report".

Stage 3. The *Leader* sends an "final oracle report" to the network members for verification. Participants verify the correctness of the signatures and then spread "final oracle report" over p2p network. As soon as the node has collected the "final oracle report" from 1/3 of the participants of the p2p network, it starts the Transmitting protocol.


Chainlink Developer Platform/Metalayer
Cryptographically Provable Data
Cryptographically Trust-Minimized Compute
Cross-Chain Value
Market Data Oracle Networks
Proof of Reserve Oracle Networks
Identity and Other Data Oracle Networks
Verifiable Randomness Function (VRF)
Automation Oracle Networks
Functions Oracle Networks
Cross-Chain Interoperability Protocol
Decentralized Oracle Networks
+ hundreds more
DeFi Contracts
CeFi Applications
Real-World Asset Proofs and Wrapped Assets
New Hybrid Smart Contracts
Web3 Contracts
Web2 Applications
DeFi Contracts
CeFi Applications
New Trust-Minimized Applications
Cross-Chain Smart Contracts
+ hundreds of blockchains
+ hundreds more
CROSS-CHAIN INTEROPERABILITY PROTOCOL
CROSS-CHAIN INTEROPERABILITY PROTOCOL
CROSS-CHAIN INTEROPERABILITY PROTOCOL
Global Banks
Asset Managers
Clearing & Settlement Depositories
Central Bank Digital Currencies